Backend Engineer Assignment — First-Party Attribution Pipeline

Oct 23, 2026 · @AD

The task

An online store wants to stop paying for third-party analytics and own its marketing data. Build the backend that collects its website events, connects them to real orders, and answers one question: **which marketing channels are actually driving revenue?**

Your service should:

1. **Receive data like a live system.** Expose `POST /collect` for website events (as a tracking pixel would send them) and `POST /webhooks/orders` for order updates (as the store's order system would send them). Write your own small sender script that replays the provided files to these endpoints in file order, with some concurrency (for example 10–20 requests at a time). No loading files straight into the database.
2. **Build the journey.** Turn raw events into visitor sessions, and work out which events and orders belong to the same person.
3. **Attribute revenue.** Credit each real order to the marketing channel that led to it using last-click attribution, and explain your attribution window. A second model (for example first click) is an optional bonus.
4. **Serve the results.** An API that returns orders, revenue, ad spend and ROAS per channel for a date range.
5. **Show it.** A simple dashboard to view those numbers by channel and by day, with a date filter. Any tool is fine (Metabase, Streamlit, Grafana, or a plain HTML page); it does not need to look polished.

This is real-world tracking data. It does not always agree with itself, and new data can keep arriving after you have processed a day. Your numbers should be ones a business could trust.

In the follow-up call, we will give you three more days of data (1–3 October) to send through your running system live, so build it to keep accepting new data rather than as a one-time script.

Everything else (design, schema, endpoints, structure) is up to you. Suggested time: 6–8 hours. Deadline: 4 days from receiving this.

What we're looking for

We are hiring someone who can own a data system end to end, not just write code that runs. This is how we will judge your work.

We look for	What that means in practice
Numbers you can defend	Your revenue and ROAS figures hold up when we check them, and you can explain where every number comes from
Judgement with messy data	You notice when the data is wrong or inconsistent, decide what to do about it, and write down why
A system, not a script	It keeps working as new data arrives, including data that changes the past
Clear thinking about scale	You know what would break first under real load and what you would change
Ownership of your code	You can explain any part of it and change it live, whether you or an AI tool wrote it
Honest communication	Your README says what works, what doesn't, and what you'd do next

What we are not looking for: a polished UI, a large number of features, or perfect code. A smaller system that is correct and well explained beats a bigger one that isn't.

The data you get

You'll receive `urbanthread_data_sep2026.zip`: one month (1–30 September 2026) of data from a fashion store, Urban Thread.

File	What it is	Size
`events.jsonl`	Website events captured by the store's tracking pixel, in the order the server received them	~56,000 events
`orders.csv`	Order webhooks from the store's order system, one row per webhook; an order gets a new webhook whenever it changes	~1,150 webhooks

File	What it is	Size
`ad_spend.csv`	Daily spend per campaign pulled from Meta, Google and TikTok	~230 rows

A short README in the zip describes each field. Treat it the way you would a new client's data: verify it before you trust it.

SQL questions

Answer these in plain SQL (PostgreSQL dialect) against your own tables, and put them in `queries.sql` with a short comment above each. They are independent of the API in the task.

1. **Sessions:** split each visitor's events into sessions, where a new session starts after 30 minutes of inactivity. Return the session count and median session length per day.
2. **Days to convert:** for each paid order, the number of days between that buyer's very first visit (on any of their devices) and the order.

Tools, submission and next steps

- **Runs on your laptop.** Everything can be built and run locally for free; no cloud account or paid tool is needed.
- **Stack:** any language and framework. SQLite is fine; PostgreSQL (free, in Docker) is also welcome.
- **Running it:** one or two commands from your README. Deploying on a free tier such as Railway is an optional bonus.
- **AI tools:** use whatever you like. You must be able to explain and change any part of your code yourself.

Send one GitHub repository link (keep your commit history) containing:

- [ ] Your code, sender script and `queries.sql`
- [ ] A README: how to run it, the decisions you made and why, and anything in the data you chose to exclude or correct
- [ ] Your final per-channel numbers for September (last click), plus a screenshot of your dashboard

- [ ] Half a page: what would break first if this ran for 500 stores at 50 million events a day, and what you would change
- [ ] A few lines on how you used AI tools, including one place where they got something wrong

Shortlisted candidates get a 45-minute call. You'll walk us through your work, then send the 1–3 October data through your running system live and show us how your numbers change. If anything here is unclear, make a sensible assumption and note it in your README.